# Assignment 14.1 - Generative Adversarial Networks (GAN)

Please submit your solution of this notebook in the Whiteboard at the corresponding Assignment entry as .ipynb-file and as .pdf.

### Please state both names of your group members here:

S M Shameem Ahmed Khan and Rashid Harvey

## Task 14.1.1: GAN for MNIST

- Implement a basic Generative Adversarial Network (GAN) model and training loop. **(RESULT)**
- Train the GAN on the MNIST dataset. **(RESULT)**
- Visualize some original and generated images from the test set. **(RESULT)**
- Visualize the embeddings of a subset (10 samples) of the test samples using `t-SNE`. You should use your trained encoder model to create those embeddings. **(RESULT)**

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# MNIST GAN Implementation

class GAN(nn.Module):
    def __init__(self, learning_rate=0.0002):
        super().__init__()

        self.generator = nn.Sequential(
            nn.Linear(256, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(True),
            nn.Linear(512, 28*28),
            nn.Softmax(dim=1)
        )

        self.discriminator = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
```

```
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

        self.learning_rate = learning_rate
        self.generator_optimizer = optim.Adam(self.generator.parameters(), lr=self.
        self.discriminator_optimizer = optim.Adam(self.discriminator.parameters(),

    def train(self, real_data_loader, epochs=50):
        loss_fn = nn.BCELoss()
        for epoch in range(epochs):
            for real_data, _ in real_data_loader:
                batch_size = real_data.size(0)
                real_data = real_data.view(batch_size, -1)

                # Train discriminator
                noise = torch.normal(mean=0.0, std=1.0, size=(batch_size, 256))
                noise = noise + 0.1 * torch.rand(batch_size, 256)
                fake_data = self.generator(noise)

                real_labels = torch.ones(batch_size, 1)
                fake_labels = torch.zeros(batch_size, 1)

                self.discriminator_optimizer.zero_grad()

                # real
                real_output = self.discriminator(real_data)
                d_loss_real = loss_fn(real_output, real_labels)
                d_loss_real.backward()

                # fake
                fake_output = self.discriminator(fake_data.detach())
                d_loss_fake = loss_fn(fake_output, fake_labels)
                d_loss_fake.backward()

                self.discriminator_optimizer.step()

                # Train generator
                self.generator_optimizer.zero_grad()
                fake_output = self.discriminator(fake_data)
                g_loss = loss_fn(fake_output, real_labels)
                g_loss.backward()
                self.generator_optimizer.step()

            print(f'Epoch [{epoch+1}/{epochs}], D Loss: {(d_loss_fake+d_loss_real).

    def generate_samples(self, num_samples=10):
        noise = torch.normal(mean=0.0, std=1.0, size=(num_samples, 256))
        noise = noise + 0.1 * torch.rand(num_samples, 256)
        fake_data = self.generator(noise)
        return fake_data.view(num_samples, 1, 28, 28).detach().numpy()
```

In [3]:
```
transform = transforms.ToTensor()
train_data = datasets.MNIST('./data', train=True, download=True, transform=transfor
test_data = datasets.MNIST('./data', train=False, download=True, transform=transfor
```

```
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)


gan = GAN(learning_rate=0.0002)
gan.train(train_loader, epochs=50)
```

```
Epoch [1/50], D Loss: 8.18304906715639e-05, G Loss: 9.445093154907227
Epoch [2/50], D Loss: 4.290732613299042e-05, G Loss: 10.07102108001709
Epoch [3/50], D Loss: 1.3000321814615745e-05, G Loss: 11.26659870147705
Epoch [4/50], D Loss: 1.8578566596261226e-06, G Loss: 13.20960521697998
Epoch [5/50], D Loss: 3.647969424491748e-05, G Loss: 10.396645545959473
Epoch [6/50], D Loss: 1.0142065548279788e-05, G Loss: 12.199474334716797
Epoch [7/50], D Loss: 6.015880444465438e-06, G Loss: 12.078620910644531
Epoch [8/50], D Loss: 5.661985596816521e-06, G Loss: 12.24538516998291
Epoch [9/50], D Loss: 1.1707545581884915e-06, G Loss: 13.681519508361816
Epoch [10/50], D Loss: 4.055214901654835 6e-07, G Loss: 14.748680114746094
Epoch [11/50], D Loss: 4.746954118672875e-07, G Loss: 14.56700325012207
Epoch [12/50], D Loss: 8.951681707003445e-08, G Loss: 16.2724666595459
Epoch [13/50], D Loss: 3.714304241952959e-08, G Loss: 17.109514236450195
Epoch [14/50], D Loss: 5.3354295204144364e-08, G Loss: 16.7593936920166
Epoch [15/50], D Loss: 8.795647232773263e-08, G Loss: 16.310317993164062
Epoch [16/50], D Loss: 2.2013967537759527e-08, G Loss: 17.633445739746094
Epoch [17/50], D Loss: 1.1979907732495576e-08, G Loss: 18.242952346801758
Epoch [18/50], D Loss: 6.638570582140346e-09, G Loss: 18.832172393798828
Epoch [19/50], D Loss: 2.123831555067568e-09, G Loss: 19.970937728881836
Epoch [20/50], D Loss: 1.932011883809537e-09, G Loss: 20.077957153320312
Epoch [21/50], D Loss: 2.2981625491524937e-09, G Loss: 19.89488410949707
Epoch [22/50], D Loss: 2.415965205671e-09, G Loss: 19.86968231201172
Epoch [23/50], D Loss: 5.685711901293189e-10, G Loss: 21.288904190063477
Epoch [24/50], D Loss: 3.7734074198603196e-10, G Loss: 21.72184181213379
Epoch [25/50], D Loss: 2.0832675862170191e-10, G Loss: 22.292724609375
Epoch [26/50], D Loss: 1.1804598964992863e-10, G Loss: 22.86157989501953
Epoch [27/50], D Loss: 9.888494190146702e-11, G Loss: 23.040294647216797
Epoch [28/50], D Loss: 6.209520397870705e-11, G Loss: 23.50281524658203
Epoch [29/50], D Loss: 4.2015921991600536e-11, G Loss: 23.893348693847656
Epoch [30/50], D Loss: 5.736740249506766 5e-11, G Loss: 23.582216262817383
Epoch [31/50], D Loss: 3.604756423913891e-11, G Loss: 24.046579360961914
Epoch [32/50], D Loss: 2.537695054094513e-11, G Loss: 24.397506713867188
Epoch [33/50], D Loss: 1.918159034386413e-11, G Loss: 24.677335739135742
Epoch [34/50], D Loss: 1.522365219996935e-11, G Loss: 24.90839195251465
Epoch [35/50], D Loss: 1.2518624158130987e-11, G Loss: 25.10399055480957
Epoch [36/50], D Loss: 1.847666637744272e-11, G Loss: 24.714794158935547
Epoch [37/50], D Loss: 1.4538344486614285e-11, G Loss: 24.9544620513916
Epoch [38/50], D Loss: 1.1961405824156834e-11, G Loss: 25.149524688720703
Epoch [39/50], D Loss: 1.0137441233681876e-11, G Loss: 25.3149471282959
Epoch [40/50], D Loss: 8.784961473551345e-12, G Loss: 25.45812225341797
Epoch [41/50], D Loss: 7.742693639012366e-12, G Loss: 25.58439826965332
Epoch [42/50], D Loss: 6.91672630628859e-12, G Loss: 25.69719123840332
Epoch [43/50], D Loss: 6.246313796059999e-12, G Loss: 25.79913330078125
Epoch [44/50], D Loss: 5.69176736991972e-12, G Loss: 25.892091751098633
Epoch [45/50], D Loss: 5.225673626457761e-12, G Loss: 25.977523803710938
Epoch [46/50], D Loss: 1.0364541690177642e-11, G Loss: 25.292823791503906
Epoch [47/50], D Loss: 8.970471067348829e-12, G Loss: 25.437227249145508
Epoch [48/50], D Loss: 7.881736929782335e-12, G Loss: 25.56660270690918
Epoch [49/50], D Loss: 7.0245944477909106e-12, G Loss: 25.681716918945312
Epoch [50/50], D Loss: 6.337185116944699e-12, G Loss: 25.784690856933594
```

Something is broken in the training, but I wasn't able to find out what's the problem, even though I tried different network architectures, etc.

# Task 14.1.2: GAN Evaluation

- Evaluate your GAN model on at least 10 generated images, e.g. via Inception Score.
  **(RESULT)**

In [ ]:
```
import torch.nn.functional as F
from transformers import AutoModelForImageClassification, AutoImageProcessor
from PIL import Image

# We are using a pretrained MNIST classifier from HF
model_name = "farleyknight/mnist-digit-classification-2022-09-04"
image_processor = AutoImageProcessor.from_pretrained(model_name)
classifier = AutoModelForImageClassification.from_pretrained(model_name)
classifier.eval()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
classifier = classifier.to(device)

print(f"Loaded pretrained classifier: {model_name}")
print(f"Labels: {classifier.config.id2label}")

def inception_score(gan_model, classifier, image_processor, n_samples=1000, n_split
    """
    Inception = exp E_x [ D_KL ( p(y|x) || p(y) ) ]
    """
    classifier.eval()

    samples = gan_model.generate_samples(num_samples=n_samples)

    # Convert to PIL
    pil_images = []
    for i in range(n_samples):
        img = (samples[i].squeeze() * 255).astype(np.uint8)
        pil_images.append(Image.fromarray(img, mode="L").convert("RGB"))

    batch_size = 64
    all_probs = []
    with torch.no_grad():
        for start in range(0, n_samples, batch_size):
            batch_imgs = pil_images[start : start + batch_size]
            inputs = image_processor(images=batch_imgs, return_tensors="pt")
            inputs = {k: v.to(device) for k, v in inputs.items()}
            logits = classifier(**inputs).logits
            probs = F.softmax(logits, dim=1).cpu().numpy()
            all_probs.append(probs)

    pyx = np.concatenate(all_probs, axis=0)  # p(y|x), shape (N, 10)

    # Split into groups and compute IS per split
    scores = []
    split_size = n_samples // n_splits
    for k in range(n_splits):
        split = pyx[k * split_size : (k + 1) * split_size]
        py = np.mean(split, axis=0, keepdims=True)
```

```
        kl = split * (np.log(split + 1e-16) - np.log(py + 1e-16))
        scores.append(np.exp(np.mean(np.sum(kl, axis=1))))

    # mean and std of IS across splits
    return float(np.mean(scores)), float(np.std(scores))

is_mean, is_std = inception_score(gan, classifier, image_processor, n_samples=1000,
print(f"\nInception Score: {is_mean:.4f} ± {is_std:.4f}")
```

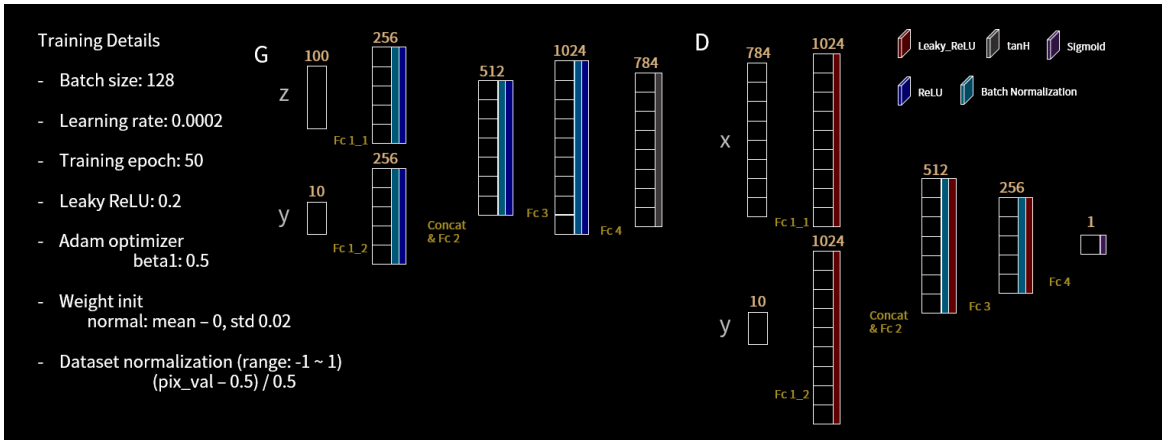
```
Loading weights: 100%|██████████| 200/200 [00:00<00:00, 223.11it/s, Materializing pa
ram=vit.layernorm.weight]
Loaded pretrained classifier: farleyknight/mnist-digit-classification-2022-09-04
Labels: {0: '0', 1: '1', 2: '2', 3: '3', 4: '4', 5: '5', 6: '6', 7: '7', 8: '8', 9:
'9'}

Inception Score: 1.0000 ± 0.0000
```

# Task 14.1.3: Conditional GAN (cGAN) (BONUS)

- Extend your previous implementations train a GAN including the label information. **(RESULT)**
- Train the cGAN on the MNIST dataset. **(RESULT)**
- Visualize some original and generated images from the test set for each class label. **(RESULT)**

In [ ]:
```
from IPython.display import Image
Image(url= "https://raw.githubusercontent.com/znxlwm/pytorch-MNIST-CelebA-cGAN-cDCG
```

Out[ ]:




In [ ]:
```
# TODO: Implement
```

# Congratz, you made it! :)